ECRformer-Light & PatchGAN Cloud Removal

The Complete "No-Black-Boxes" Question & Answer Handbook

Explained from Scratch for Beginners, 7th-Graders, and AI Researchers — With Full Annotated PyTorch Code

Covers: 1. Data & Preprocessing • 2. ECRformer Training (128×128) • 3. PatchGAN Critic • 4. Cloud Removal Inference (256×256)

Contents

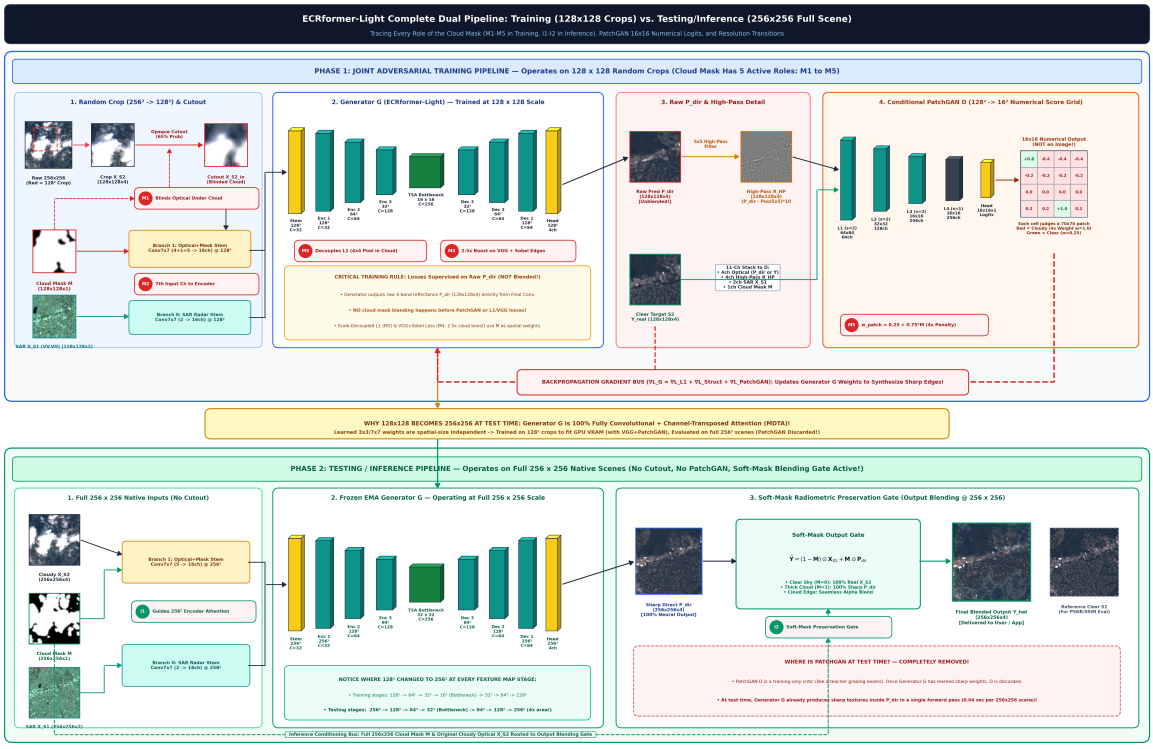
1 Part 1: The Big Picture & Satellite Basics (From Scratch) 2

2 Part 2: Data & Preprocessing (Questions & Answers) 3

3 Part 3: Training with ECRformer (Opening the Generator Black Box) 7

4 Part 4: PatchGAN & Smart Losses (Making Blurry Images Razor-Sharp) 10

5 Part 5: Final Cloud Removal — Testing & Inference at 256×256 13



Master Reference Diagram: Top = Phase 1 Joint Adversarial Training on 128×128 Crops (Cloud Mask Roles **M1**–**M5**). Bottom = Phase 2 Testing/Inference on Full 256×256 Native Scenes (Cloud Mask Roles **I1**–**I2**).

1 Part 1: The Big Picture & Satellite Basics (From Scratch)

Q1.1: What problem are we actually solving? Why do we need two different satellites?

Answer: Earth-observing satellites orbit 700 km above us taking photos of farms, rivers, forests, floods, and cities. However, at any given moment, **more than 50% of the Earth's sky is covered by clouds!**

- **Satellite #1: Sentinel-2 (The Optical Camera):** Works just like your phone camera. It captures rich colors (Red, Green, Blue, and Near-Infrared). **Problem:** Sunlight cannot pass through thick clouds. Whenever there is a cloud, Sentinel-2 only sees a giant white blob!
- **Satellite #2: Sentinel-1 (The SAR Radar Sensor):** Works like bat sonar or hospital X-rays. Instead of waiting for sunlight, it shoots its own **microwave radio beams** down to Earth and measures how the echoes bounce back. **Superpower:** Microwaves have wavelengths of ~ 5.5 cm, which are thousands of times larger than tiny cloud water droplets! So radar beams pass **straight through thick clouds, smoke, and even darkness at night** as if the clouds aren't even there!

Our Goal: Teach an AI neural network (**ECRformer**) to look at the cloudy optical photo + the cloud-piercing radar echo, and **paint back the missing colors and sharp ground structures** underneath the clouds!

7th-Grade Real-World Analogy: The Blindfolded Sculptor and the Painter

Imagine a painter (**Sentinel-2 Optical**) trying to paint a landscape, but thick fog rolls in and blocks half the town. Standing next to the painter is a sculptor with magic long sticks (**Sentinel-1 SAR Radar**) who can reach right through the fog and feel the exact shape of every roof, road, river, and tree! **ECRformer** is the translator who listens to what the radar sculptor feels and tells the painter exactly what colors and sharp edges to draw inside the fog!

Q1.2: What do VV and VH mean in Sentinel-1 SAR Radar?

Answer: SAR stands for **Synthetic Aperture Radar**. When the satellite shoots a microwave pulse, the wave vibrates vertically (**V**). When it hits the ground and bounces back, two things can happen:

1. **VV (Vertical Send → Vertical Receive):** Measures **surface roughness**. Calm water is smooth like a mirror, so the radar beam bounces away and never returns (looks pitch black). Concrete buildings and rocky ground are rough, so they bounce a strong echo straight back (looks bright).
2. **VH (Vertical Send → Horizontal Receive):** Measures **complex 3D volume scattering** (like tree branches and crop leaves). When a vertical wave hits a forest canopy, the leaves twist the wave so it bounces back horizontally!

Together, **VV + VH (2 channels)** give the AI a complete geometric blueprint of water, buildings, roads, and forests underneath any cloud!

2 Part 2: Data & Preprocessing (Questions & Answers)

Q2.1: In the dataset (`fall` subset), is the data saved as images (`.png`/`.jpg`) or numbers?

Answer: It is saved **100% in numerical format** as compressed NumPy archive files (`.npz`), **NOT** as `.png` or `.jpg` images!
Every single `.npz` file on your hard drive (e.g., `ROIs1970_fall_s2_119_p100.npz`) is a zip-like container holding 5 arrays of 32-bit decimal numbers (`float32`):

Key in `.npz`	Array Shape	Number Range	What the Numbers Represent
`s2_cloudy`	$[256, 256, 4]$	0.0 to 1.0	Cloudy optical reflectance (Red, Green, Blue, NIR)
`s1`	$[256, 256, 2]$	-35.0 to 0.0 dB	Radar backscatter in decibels (VV, VH)
`s2_clear`	$[256, 256, 4]$	0.0 to 1.0	Cloud-free ground truth optical reflectance
`mask`	$[256, 256]$	0.0 to 1.0	Cloud probability mask ($0 =$ Clear, $1 =$ Cloud)
`cf`	1 number	0.0 to 1.0	Total fraction of the patch covered by clouds

Q2.2: Why can't we just store the satellite data as normal `.jpg` or `.png` pictures?

Answer: Three big reasons:

1. **Too Many Channels:** Normal `.jpg`/`.png` images only hold 3 color channels (Red, Green, Blue). Our optical satellite has **4 channels** (including invisible Near-Infrared **B08**, which plants reflect strongly), plus **2 radar channels** = 6 channels total!
2. **Negative Numbers in Radar:** Radar echoes are measured in **decibels (dB)**, which are negative numbers like -22.4 dB or -11.8 dB. Normal image files can only store whole positive integers from 0 to 255.
3. **Precision Loss:** Standard images crush smooth physical measurements into 256 steps (0–255). Scientific AI needs exact floating-point decimals like 0.08472 to measure true surface reflectance.

Q2.3: How do we find the Cloud Mask (`M`) in preprocessing? How does the computer separate White (Cloud) from Black (Clear Land)?

Answer: This is a classic trap! You might think: *"Just check if a pixel looks bright white!"* **Why that fails:** If you only check for bright white pixels, then **snow on mountains, white concrete rooftops, desert sand, and beaches** will all be falsely marked as clouds!

Opening the Black Box: How `s2cloudless` Detects Real Clouds Using Invisible Light

During preprocessing (`rebuild_cache_v3.py`), we use the official European Space Agency (ESA) machine-learning cloud detector called **`s2cloudless`** (`S2PixelCloudDetector`). It inspects **all 13 wavelengths** of Sentinel-2, especially two invisible infrared bands:

- **The Cirrus Band (Band 10, 1375 nm):** Water vapor in the lower atmosphere absorbs 100% of sunlight at this wavelength. That means the Earth's ground (even white snow or white roofs!) looks **pitch black** in Band 10. Only high-altitude clouds sit above the water vapor and reflect Band 10 brightly!
- **Short-Wave Infrared (SWIR, Bands 11 & 12):** Desert sand and white buildings reflect SWIR strongly, while water droplets inside clouds absorb SWIR differently.

`s2cloudless` outputs a probability from 0.0 (0% chance of cloud) to 1.0 (100% chance of cloud) for every pixel:

- **White in Mask ($\mathbf{M} = 1.0$):** Cloud probability $> 40\%$. The optical view is blocked!
- **Black in Mask ($\mathbf{M} = 0.0$):** Clear sky ($< 40\%$). The ground is clearly visible!
- **2-Pixel Dilation Border:** We expand the white cloud mask outward by 2 pixels (`dilation_size=2`) so thin, semi-transparent haze around cloud edges is also caught!

```python
from s2cloudless import S2PixelCloudDetector
import tifffile as tiff
import numpy as np

# 1. Initialize ESA's multi-spectral cloud detector
detector = S2PixelCloudDetector(
    threshold=0.4,       # Any pixel with > 40% cloud probability becomes Cloud (1.0)
    average_over=4,      # Smooths probabilities over a 4x4 window to remove noise
    dilation_size=2,     # Expands cloud border by 2 pixels to catch thin edge haze
    all_bands=True       # Uses all 13 Sentinel-2 spectral bands (including Cirrus & SWIR)
)

# 2. Load raw 16-bit GeoTIFF satellite numbers (0 to 10,000) and scale to [0.0, 1.0]
cloudy_img = tiff.imread("s2_cloudy_scene.tif")
scaled = np.expand_dims(cloudy_img.astype(np.float32) / 10000.0, axis=0)

# 3. Predict cloud probability map [256, 256] in [0.0, 1.0]
prob_map = detector.get_cloud_probability_maps(scaled)[0]

# 4. Separate into White (1.0 = Cloud) and Black (0.0 = Clear Land)
mask = (prob_map > 0.4).astype(np.float32)
```

Listing 1: Exact Preprocessing Code That Generates the Cloud Mask (from `rebuild_cache_v3.py`)

Q2.4: How do we preprocess the `.npz` numbers to feed them into ECRformer?

Answer: Before feeding a patch into ECRformer, our dataset loader (`CachedSEN12MSCRDataset` in `backend/data/cache_dataset.py`) performs **4 steps**:

1. **Channel Reordering (`H, W, C` → `C, H, W`):** NumPy stores arrays as `[Height, Width, Channels]` ($[256, 256, 4]$), whereas PyTorch neural networks require `[Channels, Height, Width]` ($[4, 256, 256]$).

2. **SAR Radar Normalization:** Raw radar decibels are negative (VV $\in [-25, 0]$ dB, VH $\in [-35, 0]$ dB). Neural networks work best when all inputs are between 0.0 and 1.0. So we linearly rescale them:

$$\mathrm{VV}_{\mathrm{norm}} = \frac{\mathrm{clamp}(\mathrm{VV}, -25, 0) + 25}{25}, \qquad \mathrm{VH}_{\mathrm{norm}} = \frac{\mathrm{clamp}(\mathrm{VH}, -35, 0) + 35}{35}$$

3. **Soft Cloud Mask Smoothing:** We apply a tiny 5×5 average pool to the dilated binary mask so the edge between 0.0 (black) and 1.0 (white) has a smooth feathering ramp $(0.2, 0.5, 0.8)$.

4. **Random 128×128 Crop (During Training Only!):** During training, we slice the exact same random 128×128 square window out of `s2_cloudy`, `s1`, `s2_clear`, and `mask`. During testing, we keep the full 256×256 size!

```python
def __getitem__(self, idx):
    data = np.load(self.files[idx])
    # 1. Load float32 numerical arrays and transpose [H, W, C] -> [C, H, W]
    s2_cloudy_t = torch.from_numpy(data['s2_cloudy'].astype(np.float32).transpose(2, 0, 1))  # [4, 256, 256]
    s1_t        = torch.from_numpy(data['s1'].astype(np.float32).transpose(2, 0, 1))         # [2, 256, 256]
    s2_clear_t  = torch.from_numpy(data['s2_clear'].astype(np.float32).transpose(2, 0, 1))   # [4, 256, 256]
    mask_t      = torch.from_numpy(data['mask'].astype(np.float32)).unsqueeze(0)             # [1, 256, 256]

    # 2. SAR Radar Normalization from negative decibels (dB) to [0.0, 1.0]
    vv = (torch.clamp(s1_t[0:1], -25.0, 0.0) + 25.0) / 25.0
    vh = (torch.clamp(s1_t[1:2], -35.0, 0.0) + 35.0) / 35.0
    s1_t = torch.cat([vv, vh], dim=0)

    # 3. Random 128x128 crop during Training (None = full 256x256 during Testing!)
    if self.crop_size is not None:
        top = torch.randint(0, 256 - self.crop_size + 1, (1,)).item()
        left = torch.randint(0, 256 - self.crop_size + 1, (1,)).item()
        s2_cloudy_t = s2_cloudy_t[:, top:top+128, left:left+128]
        s1_t        = s1_t[:, top:top+128, left:left+128]
        s2_clear_t  = s2_clear_t[:, top:top+128, left:left+128]
        mask_t      = mask_t[:, top:top+128, left:left+128]

    return s2_cloudy_t, s1_t, s2_clear_t, cf, mask_t
```

Listing 2: Exact Data Loading & Preprocessing Code (from `backend/data/cache_dataset.py`)

Q2.5: What is "Opaque Cloud Cutout" (Cloud Mask Role `M1`) during training, and why does the AI try to "cheat" without it?

Answer: In satellite datasets, about half of the "cloudy" photos are actually **thin, semi-transparent haze** where you can still faintly see the outlines of roads and fields underneath.

- **How the AI Cheats:** Neural networks are lazy! If the AI can dimly see road edges through

thin optical haze, it learns a shortcut: *"Just boost the contrast of the cloudy optical photo and completely ignore the Sentinel-1 SAR radar!"* Then, when tested on a **100% thick opaque white cloud**, the AI fails completely and outputs a blurry smudge!

- **How Role `M1` (`apply_opaque_cloud_cutout`) Stops Cheating:** During training, on 65% of patches, we use the Cloud Mask **M** to **intentionally blind** the optical image inside the cloud (applying a heavy 29×29 blur veil over the cloudy pixels). Because all optical edges inside **M** are wiped out, the AI is **forced** to read the Sentinel-1 SAR radar channels to reconstruct the ground!

```python
def apply_opaque_cloud_cutout(s2_c: torch.Tensor, mask: torch.Tensor, prob: float = 0.65):
    # Two passes of 15x15 average pooling erase all sharp ground edges inside the cloud
    veil = F.avg_pool2d(F.pad(s2_c, (7, 7, 7, 7), mode="reflect"), kernel_size=15, stride=1)
    veil = F.avg_pool2d(F.pad(veil, (7, 7, 7, 7), mode="reflect"), kernel_size=15, stride=1)

    # Cloud Mask M (Role M1) replaces cloudy pixels with the featureless veil!
    alpha = torch.clamp((mask - 0.15) / 0.85, 0.0, 1.0) * do_aug
    s2_aug = (1.0 - alpha) * s2_c + alpha * veil
    return torch.clamp(s2_aug, 0.0, 1.0)
```

Listing 3: Opaque Cloud Cutout Anti-Cheating Augmentation (from `backend/losses/cloud_structure_loss.py`)

3 Part 3: Training with ECRformer (Opening the Generator Black Box)

Q3.1: What is a "Convolution" and what is a "Channel"? (Explained for a 7th Grader)

Answer:

- **What is a Convolution (3×3 Kernel)?** Imagine sliding a tiny 3×3 magnifying glass across a grid of numbers, one step at a time. Inside the magnifying glass are 9 learned weights. At each spot, it multiplies the 9 pixel numbers by its 9 weights and adds them up to detect a specific pattern (like a vertical road edge, a corner, or a bumpy texture).
- **What is a Channel (C)?** Think of channels as **transparent plastic sheets stacked on top of each other**, each highlighting one specific feature across the map:
  - Sheet #1 might light up wherever there is **water**.
  - Sheet #2 might light up wherever there are **sharp building corners in radar**.
  - Sheet #3 might light up wherever there is **green forest chlorophyll**.

  When we say a layer has $C = 64$ channels, it means the network is maintaining **64 different feature maps** describing the terrain simultaneously!

Q3.2: Why does ECRformer have TWO separate front doors ("Decoupled Stem") and why does the Cloud Mask enter here (Role `M2`)?

Answer: Optical photos (sunlight reflectance) and SAR Radar (microwave echoes) obey completely different laws of physics! If we mashed them into a single convolution right at the start, their raw numbers would clash.
Instead, `ECRformerAdaptedModel` uses a **Decoupled Two-Branch Stem**:

1. **Branch 0 (Radar Door):** A 7×7 convolution that takes **2 SAR channels** (VV, VH) and extracts **16 radar feature channels**.
2. **Branch 1 (Optical + Mask Door — Role `M2`):** A 7×7 convolution that takes **4 Optical channels + 1 Cloud Mask channel = 5 channels** and extracts **16 optical feature channels**.
3. **Fusion (`Concat`):** Once both sensors are translated into a common language of 16 feature maps each, we stack them together into $16 + 16 = 32$ **channels**!

Why feed Cloud Mask `M2` into Branch 1? So from the very first layer, every neuron in the optical branch knows: *"Wait! The mask says* **M** $= 1.0$ *here—do NOT trust the white optical pixels at this spot; wait for Branch 0's radar features instead!"*

Q3.3: How does the Hourglass (Encoder → Bottleneck → Decoder) and Skip Connections work inside ECRformer?

Answer: Look at Box 2 in the Master Diagram. ECRformer is shaped like an **hourglass (U-Net)**:

1. **The Encoder (Left Half — Shrinking Size, Growing Channels):** Starts at 128×128 (32 channels) → shrinks to 64×64 (64 channels) → shrinks to 32×32 (128 channels) → reaches the narrow center (**Bottleneck**) at 16×16 (256 channels). *How does it shrink without losing data?* Using `PixelUnshuffle`: it takes every 2×2 square of pixels and folds them

into 4 separate channels!

2. **The TSA Bottleneck (Center):** At 16×16 with 256 rich feature channels, the network uses a **Transformer (MDTA)** to connect global information across the entire scene.
3. **The Decoder (Right Half — Expanding Back to Full Size):** Uses `PixelShuffle` to unfold channels back into spatial pixels: $16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128$, ending with a Final 3×3 Convolution that outputs **4 optical channels** ($\mathbf{P}_{\text{dir}}$).
4. **Skip Connections (The Overhead Bridges):** When you shrink an image down to 16×16, tiny 1-pixel road positions get blurry. So we build **3 overhead bridges (Skip 0, Skip 1, Skip 2)** that copy the high-resolution maps from the Encoder directly across to the Decoder!

Q3.4: BLACK BOX EXPLAINED — What is MDTA (Channel-Based Attention vs. Pixel-Based Attention), and how does it let a 128×128 trained model run on 256×256 test images?

Answer: This is one of the most important concepts in modern Computer Vision!

Opening the Black Box: Pixel-Based Attention vs. Channel-Based Attention (MDTA)

1. Standard Transformer (Pixel-Based Attention — "WHERE is it?"): In a normal Vision Transformer (ViT), every single pixel compares itself to every other pixel.

- If a feature map has size $H \times W$, there are $N = H \cdot W$ pixels.
- The attention table has size $(H \cdot W) \times (H \cdot W)$.
- If you train on 128×128 (where the bottleneck is $16 \times 16 = 256$ pixels) and then test on 256×256 (where the bottleneck is $32 \times 32 = 1{,}024$ pixels), the attention table jumps by **16× in size** and fixed positional embeddings **crash with a shape error**!

2. ECRformer's MDTA (Channel-Based Attention — "WHAT feature correlates with WHAT feature?"): Instead of comparing pixel coordinates (x_1, y_1) to (x_2, y_2), **Multi-DConv Head Transposed Attention (MDTA)** flips (transposes) the matrix multiplication so it compares **Channel to Channel**!

- Imagine Channel #14 detects **"Radar Building Corners"** and Channel #52 generates **"Red Clay Roof Color"**.
- MDTA computes the dot-product across all pixels between Channel #14 and Channel #52:

$$\mathbf{A}_{\text{channel}} = \mathbf{Q} \cdot \mathbf{K}^T \in \mathbb{R}^{C \times C}$$

- Look at the shapes: $\mathbf{Q}$ has shape $[C, H \cdot W]$ and $\mathbf{K}^T$ has shape $[H \cdot W, C]$. When you multiply $[C, HW] \times [HW, C]$, the spatial size HW **cancels out (is summed over)**, leaving an Attention Matrix of shape $[C, C]$ (256×256 channels)!

Why $128 \times 128 \rightarrow 256 \times 256$ works automatically: Because every convolution is a sliding 3×3 window (which can slide over any size grid) and MDTA's attention matrix is always $[C, C]$ regardless of H and W, **not a single weight inside ECRformer depends on the image height or width!** You can train on 128×128 crops to save GPU memory and test on 256×256 (or 512×512) with **zero code changes**!

```
class TransposedAttention(nn.Module):
    """Multi-DConv Head Transposed Self-Attention (MDTA)."""
```

```python
    def forward(self, x):
        b, c, h, w = x.shape
        q, k, v = self.qkv_dwconv(self.qkv(x)).chunk(3, dim=1)

        # Reshape each channel into a flat vector of length (h * w)
        q = rearrange(q, 'b (head c) h w -> b head c (h w)', head=self.num_heads)
        k = rearrange(k, 'b (head c) h w -> b head c (h w)', head=self.num_heads)
        v = rearrange(v, 'b (head c) h w -> b head c (h w)', head=self.num_heads)

        q = torch.nn.functional.normalize(q, dim=-1)
        k = torch.nn.functional.normalize(k, dim=-1)

        # MAGIC STEP: [c, h*w] @ [h*w, c] -> [c, c] Channel Attention Matrix!
        # Notice (h*w) disappears! Works identically for 128x128 or 256x256!
        attn = (q @ k.transpose(-2, -1)) * self.temperature
        attn = attn.softmax(dim=-1)

        out = (attn @ v) # [c, c] @ [c, h*w] -> [c, h*w]
        return self.project_out(rearrange(out, 'b head c (h w) -> b (head c) h
            w', head=self.num_heads, h=h, w=w))
```

Listing 4: Exact MDTA Channel-Transposed Attention Code (from `backend/models/module.py`)

Q3.5: What does ECRformer output during training, and do we blend the Cloud Mask before checking the training losses?

Answer:

Key Golden Rule: No Blending Before Training Losses!

During training, ECRformer's final 3×3 convolution outputs `pred_direct` ($\mathbf{P}_{\text{dir}}$), a raw $128 \times 128 \times 4$ optical prediction. **We pass `pred_direct` DIRECTLY into all training losses and PatchGAN—we do NOT blend the cloud mask before PatchGAN!** Why? Because if we blended $(1 - \mathbf{M})\mathbf{X}_{\text{cloudy}}$ before calculating the loss, the clear areas would automatically have zero error, and the neural network would get lazy and output garbage in clear regions! By grading `pred_direct` directly, we force ECRformer to learn to paint a complete, seamless satellite photo everywhere.

4 Part 4: PatchGAN & Smart Losses (Making Blurry Images Razor-Sharp)

Q4.1: Why do normal AI cloud-removal models produce blurry, smudged images?

Answer: It is caused by two classic traps when using simple pixel-by-pixel $\mathcal{L}_1$ loss ($|\text{Predicted} - \text{Target}|$):

1. **The "Safe Average" Trap:** Under a thick cloud, the radar says *"there is a field of trees here."* Could the exact tree leaves be dark green (0.05), medium green (0.10), or bright green (0.15)? If the AI guesses a sharp texture and is off by 1 pixel, $\mathcal{L}_1$ punishes it heavily. So the AI learns a cowardly trick: **paint a flat, blurry average** (0.10) to get the lowest average penalty!

2. **The "Double-Edge Penalty":** Sentinel-1 radar and Sentinel-2 optical satellites fly in different orbits, so a road edge in radar might be shifted by **1 pixel** compared to the optical photo. If the AI draws a razor-sharp road shifted by 1 pixel, pixel-by-pixel $\mathcal{L}_1$ punishes it **twice** (once where the road was supposed to be, and once where the AI drew it). To avoid this double punishment, standard $\mathcal{L}_1$ forces the AI to smear the road into a blurry gray band!

Q4.2: How do Cloud Mask Roles `M3` (Scale-Decoupled L1) and `M4` (VGG + Sobel Boost) solve the blur trap?

Answer:

- **Role `M3` — Scale-Decoupled L1 (`compute_scale_decoupled_l1`):** We use the Cloud Mask **M** to change how $\mathcal{L}_1$ grades the image:
  - **Outside Clouds ($\mathbf{M} = 0$):** Apply strict 1×1 pixel-by-pixel $\mathcal{L}_1$ so clear land stays pixel-exact.
  - **Inside Clouds ($\mathbf{M} = 1$):** Turn off 80% of the 1×1 pixel $\mathcal{L}_1$ and replace it with 4×4 **and** 8×8 **average-pooled** $\mathcal{L}_1$! This forces the AI to get the **average regional color right** inside the cloud, while freeing it from the 1-pixel double-edge penalty so it is allowed to draw sharp edges!

- **Role `M4` — VGG Perceptual + Sobel Edge Loss (`CloudStructurePerceptualLoss`):** Measures edge sharpness (Sobel gradients) and deep texture features (VGG-16) and multiplies the penalty by `cloud_boost = 2.5`$\times$ inside the Cloud Mask **M**.

```python
def compute_scale_decoupled_l1(pred_direct, target, mask, projs=None):
    # 1. Clear region (1 - 0.8*M): strict 1x1 pixel L1
    l1_1x1_clear = (torch.abs(pred_direct - target) * (1.0 - 0.80 * mask)).mean()

    # 2. Cloudy region (M): 4x4 and 8x8 pooled L1 checks regional color, NOT
        1-pixel shift!
    p_4 = F.avg_pool2d(pred_direct, kernel_size=4, stride=2, padding=1)
    t_4 = F.avg_pool2d(target,      kernel_size=4, stride=2, padding=1)
    m_4 = F.avg_pool2d(mask,        kernel_size=4, stride=2, padding=1)
    l1_low_4 = (torch.abs(p_4 - t_4) * (1.0 + m_4)).mean()
    ...
```

Listing 5: Scale-Decoupled L1 Loss Using Cloud Mask Role M3 (from `backend/losses/patchgan_loss.py`)

Q4.3: BLACK BOX EXPLAINED — What is PatchGAN (D), what exact inputs does it take, and is its output an image or numbers?

Answer:

7th-Grade Real-World Analogy: The Art Forger (Generator G) vs. The Art Detective (PatchGAN D)

Think of **ECRformer** (G) as an **Art Forger** trying to paint realistic terrain over a cloud hole, and **PatchGAN** (D) as a strict **Art Detective** with a magnifying glass.

- If the Forger paints a blurry smudge inside the cloud, the Detective immediately yells: *"FAKE! Real satellite photos have crisp rivers and sharp building roofs, not blurry smudges!"*
- Crucial Point: **The Detective (PatchGAN) NEVER holds a paintbrush!** PatchGAN does **NOT** output an image. It outputs a **scorecard of numbers** grading whether each patch looks Real or Fake!

Opening the Black Box: Inside the 11-Channel Conditional PatchGAN Discriminator

1. What exact inputs go into PatchGAN (D)? We feed an **11-channel stack** of size 128×128 into PatchGAN:

- **4 Channels — Optical Image:** Either ECRformer's raw prediction $\mathbf{P}_{\text{dir}}$ (when grading Fake) OR the real clear satellite target $\mathbf{Y}_{\text{real}}$ (when grading Real).
- **4 Channels — 5×5 High-Pass Edge Residual ($\mathbf{R}_{\mathbf{HP}}$):** Computed as $(\text{Optical} - \text{AvgPool}_{5\times5}(\text{Optical})) \times 10$. This subtracts smooth colors leaving **only fine edges and textures**. If ECRformer predicted a blurry cloud hole, $\mathbf{R}_{\text{HP}}$ is flat zero there, making blur instantly obvious to the Detective!
- **2 Channels — Sentinel-1 SAR Radar ($\mathbf{X}_{S1}$):** So the Detective can check if the painted roads actually match the radar blueprint!
- **1 Channel — Cloud Mask (M):** Tells the Detective where the cloud hole was.

2. What comes OUT of PatchGAN (D)? PatchGAN passes the $128 \times 128 \times 11$ stack through three strided convolutions ($\div 2 \to \div 2 \to \div 2$):

$$128{\times}128 \xrightarrow{\text{Layer 1}} 64{\times}64 \xrightarrow{\text{Layer 2}} 32{\times}32 \xrightarrow{\text{Layer 3}} 16{\times}16 \xrightarrow{\text{Head}} \mathbf{16 \times 16 \times 1} \text{ Matrix of Numbers!}$$

- The output is a 16×16 **grid of 256 decimal numbers (logits)—NOT an image!**
- Each single number in that $16{\times}16$ grid is the Detective's grade for a $70{\times}70$ **overlapping square patch** of the 128×128 image:
  - **Positive Number (e.g., $+1.2$):** *"This 70×70 patch looks sharp and REAL!"*
  - **Negative Number (e.g., -0.9):** *"This 70×70 patch is blurry and FAKE!"*

Q4.4: Does PatchGAN sharpen the whole image or just the blurry cloudy area? How does Cloud Mask Role M5 work, and how does the feedback return to ECRformer?

Answer:

1. **How Cloud Mask Role M5 Focuses PatchGAN on the Blurry Cloudy Area:** We shrink the Cloud Mask $\mathbf{M}$ down to 16×16 to match PatchGAN's 16×16 scorecard grid, and calculate a weight for every cell:

$$w_{\text{patch}} = 0.25 + 0.75 \times \mathbf{M}_{\text{down}}$$

   - Where the sky was **Clear** ($\mathbf{M} = 0$): Weight = **0.25** (light check).
   - Where the sky was **Cloudy** ($\mathbf{M} = 1$): Weight = **1.00** ($4\times$ **stronger penalty!**).

   So PatchGAN focuses $4\times$ **harder** on forcing sharp textures inside the blurry cloud region!

2. **How Does PatchGAN Make ECRformer Sharper Without Touching the Image?** During training (`loss_g.backward()`), the 16×16 scorecard numbers are combined with $\mathcal{L}_1$ and VGG losses into a single error signal. PyTorch sends **Backpropagation Gradients** ($\nabla \mathcal{L}_G$) backward (the red dashed arrow at the bottom of Phase 1 in the Master Diagram) straight into **ECRformer's convolution weights**. Iteration after iteration, ECRformer's own weights learn how to draw crisp buildings, roads, and rivers so the Detective stops giving it negative scores!

```python
class ConditionalPatchGANDiscriminator(nn.Module):
    @staticmethod
    def _build_input(opt: torch.Tensor, s1: torch.Tensor, mask: torch.Tensor):
        # Extract 5x5 High-Pass Edge Residual: (Image - Blurred_Image) * 10
        low = F.avg_pool2d(F.pad(opt.float(), (2, 2, 2, 2), mode="reflect"),
            kernel_size=5, stride=1)
        high_pass = (opt.float() - low) * 10.0
        # Stack 11 channels: 4ch Optical + 4ch High-Pass + 2ch SAR + 1ch Mask
        return torch.cat([opt.float() * 3.0, high_pass, s1.float(),
            mask.float()], dim=1)

    def forward(self, opt, s1, mask):
        x = self._build_input(opt, s1, mask)          # [B, 11, 128, 128]
        f1 = self.layer1(x)                           # [B, 64, 64, 64]
        f2 = self.layer2(f1)                          # [B, 128, 32, 32]
        f3 = self.layer3(f2)                          # [B, 256, 16, 16]
        f4 = self.layer4(f3)                          # [B, 256, 16, 16]
        logits = self.head(f4)                        # [B, 1, 16, 16] -> 16x16
            Numerical Grid!
        return logits, [f1, f2, f3, f4]

def compute_g_gan_and_fm_loss(disc, pred_direct, target, s1, mask, w_adv=0.015,
    w_fm=0.08):
    fake_logits, fake_feats = disc(pred_direct, s1, mask)
    # Role M5: Downsample Cloud Mask to 16x16 -> 4x stronger weight (1.0 vs
        0.25) on cloudy patches!
    m_down = F.interpolate(mask.float(), size=fake_logits.shape[-2:],
        mode="bilinear")
    w_patch = 0.25 + 0.75 * m_down
    loss_adv = -(fake_logits * w_patch).mean()
    return w_adv * loss_adv + w_fm * loss_fm
```

Listing 6: Conditional PatchGAN & Cloud-Mask Weighted Loss Role M5 (from `backend/losses/patchgan_loss.py`)

5 Part 5: Final Cloud Removal — Testing & Inference at 256×256

Q5.1: When and at which exact stage does 128×128 become 256×256? And what happens to PatchGAN during testing?

Answer:

- **When 128×128 Becomes 256×256:** As soon as training finishes! During evaluation/testing (Phase 2, Bottom of Master Diagram), we turn off random cropping and feed the **full** 256×256 **native satellite scene** directly into ECRformer. Every single layer inside ECRformer now processes $4\times$ the spatial area:

ECRformer Layer / Stage	Channels (C)	Training Shape (128^2 Crop)	Testing Shape (256^2 Full Scene)
1. Multimodal Input ($\mathbf{X}_{S1}, \mathbf{X}_{S2}, \mathbf{M}$)	7	$[B, 7, 128, 128]$	$[B, 7, 256, 256]$
2. Decoupled Stem + Encoder 1	32	$[B, 32, 128, 128]$	$[B, 32, 256, 256]$
3. Encoder Stage 2 (`Enc 2`)	64	$[B, 64, 64, 64]$	$[B, 64, 128, 128]$
4. Encoder Stage 3 (`Enc 3`)	128	$[B, 128, 32, 32]$	$[B, 128, 64, 64]$
5. **TSA Bottleneck (MDTA)**	256	$[\mathbf{B}, \mathbf{256}, \mathbf{16}, \mathbf{16}]$	$[\mathbf{B}, \mathbf{256}, \mathbf{32}, \mathbf{32}]$
6. Decoder Stages $3 \to 2 \to 1$	$128 \to 64$	$32^2 \to 64^2 \to 128^2$	$64^2 \to 128^2 \to 256^2$
7. Raw Direct Output ($\mathbf{P}_{\text{dir}}$)	4	$[B, 4, 128, 128]$	$[B, 4, 256, 256]$
8. **PatchGAN Critic (D)**	$11 \to 1$	$[B, 1, 16, 16]$ (Score Grid)	**REMOVED (Not Used!)**
9. **Soft-Mask Blended Output ($\hat{\mathbf{Y}}$)**	4	*Not Used in Loss*	$[\mathbf{B}, \mathbf{4}, \mathbf{256}, \mathbf{256}]$ (Final Output)

- **Where is PatchGAN at Test Time? Completely removed!** Once the Art Forger (ECRformer) has graduated and learned sharp weights (`best_model.pth`), we don't need the Art Detective (PatchGAN) anymore. ECRformer generates sharp 256×256 textures on its own in a single forward pass (0.04 seconds)!

Q5.2: How does the Cloud Mask work during Testing (Roles `I1` and `I2`), and why do we do Soft-Mask Blending at the very end?

Answer: At test time, the 256×256 Cloud Mask $\mathbf{M}$ performs **2 jobs**:

1. **Role `I1` (Input Guide):** Enters Branch 1 (`Conv7x7(5 -> 16)`) so the encoder knows which pixels of the 256×256 scene are covered by clouds.

2. **Role `I2` (Soft-Mask Radiometric Preservation Gate at Output):** Once ECRformer outputs its raw 256×256 prediction $\mathbf{P}_{\text{dir}}$, we apply the blending gate equation:

$$\hat{\mathbf{Y}} = (1.0 - \mathbf{M}) \odot \mathbf{X}_{\text{cloudy}} + \mathbf{M} \odot \mathbf{P}_{\text{dir}}$$

   - **In Clear-Sky Pixels ($\mathbf{M} = 0.0$):** $(1-0)\mathbf{X}_{\text{cloudy}} + 0 \cdot \mathbf{P}_{\text{dir}} = \mathbf{X}_{\text{cloudy}}$. We keep **100% of the authentic real satellite pixels**! Why replace a real photograph of clear land with an AI guess?
   - **In Thick-Cloud Pixels ($\mathbf{M} = 1.0$):** $(1-1)\mathbf{X}_{\text{cloudy}} + 1 \cdot \mathbf{P}_{\text{dir}} = \mathbf{P}_{\text{dir}}$. We replace the white cloud 100% with ECRformer's sharp radar-guided reconstruction!
   - **At Cloud Borders ($\mathbf{M} = 0.3 \sim 0.7$):** Smoothly feathers the transition so there are **zero visible seams or cutout lines**!

```python
# 1. Forward pass inside ECRformerAdaptedModel (backend/models/ecrformer_adapted.py)
def forward(self, s2_cloudy, s1, mask):
    # Role I1: Concatenate SAR (2ch) + Cloudy Optical (4ch) + Cloud Mask (1ch) = 7 channels
    x_in = torch.cat([s1, s2_cloudy, mask], dim=1)      # [1, 7, 256, 256]

```

```
    # Generator G predicts raw 4-band optical reflectance P_dir
    pred_direct, projs = self.backbone(x_in)           # [1, 4, 256, 256]

    # Role I2: Soft-Mask Radiometric Preservation Gate
    out = (1.0 - mask) * s2_cloudy + mask * pred_direct
    return out, pred_direct, projs

# 2. Convert floating-point reflectance tensor [1, 4, 256, 256] into a visible RGB .png photo
def save_satellite_rgb_png(out_tensor, save_path="cloud_free_result.png"):
    # Take first 3 channels (Red=B04, Green=B03, Blue=B02) and transpose (C,H,W) -> (H,W,C)
    rgb = out_tensor[0, :3].detach().cpu().numpy().transpose(1, 2, 0) # [256, 256, 3]
    # Multiply by 3.0x brightness (since natural Earth reflectance is ~0.10) and scale to 0..255
    rgb_8bit = (np.clip(rgb * 3.0, 0.0, 1.0) * 255.0).round().astype(np.uint8)
    Image.fromarray(rgb_8bit).save(save_path)
```

Listing 7: Complete Inference Forward Pass, Soft-Mask Blending (Role I2), and PNG Image Conversion